

SIMATS ENGINEERING

ASSESSMENT TOOL 2

SCENARIO-BASED ASSIGNMENT

Course Name: Artificial Intelligence

Course Code: CSA1717

Course Outcome: CO2

Submitted By:

Name: Abinaya Sri J

Register Number: 192524429

QUESTION 1

AI-Powered Drone Navigation for Emergency Medicine Delivery

1.1 Scenario Understanding

A government uses AI-powered drones to deliver emergency medicines to people in flood-affected areas. The environment is dynamic because roads may become blocked and weather conditions can change travel costs. The drone receives partial real-time information and must quickly select a safe and efficient route.

1.2 Problem Statement

The drone must reach the destination safely and quickly while considering blocked paths, terrain conditions, weather and travel costs. The main challenge is selecting an appropriate search algorithm for route planning under uncertain and changing conditions.

1.3 Objectives

- To understand Greedy Best-First Search in a dynamic environment.
- To explain the working of A* Search.

- To analyze the effect of heuristic accuracy.
- To study the effect of blocked paths on search performance.
- To recommend the most suitable algorithm for emergency delivery.

1.4 Greedy Best-First Search

Greedy Best-First Search selects the node that appears closest to the destination based on the heuristic value.

Working:

1. Estimate the distance from the current node to the destination.
2. Select the node with the smallest heuristic value.
3. Continue until the destination is reached.

Strengths:

- Fast route selection.
- Simple to implement.
- Useful when quick decisions are required.
- Generally explores fewer nodes.

Weaknesses:

- Ignores the actual path cost.
- May select a costly or risky route.
- Does not guarantee the optimal path.
- A blocked route may require replanning.

1.5 A* Search

A* Search considers both the cost already travelled and the estimated cost to the destination.

Formula:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = actual cost from the starting point.
- $h(n)$ = estimated cost from the current node to the destination.
- $f(n)$ = total estimated path cost.

Working:

1. Start from the source node.
2. Calculate the actual cost, $g(n)$.
3. Calculate the heuristic value, $h(n)$.
4. Calculate $f(n)$.
5. Select the node with the lowest $f(n)$.
6. Continue until the destination is reached.

A* is more suitable when both route cost and distance need to be considered.

1.6 Impact of Heuristic Accuracy

Heuristic Condition	Greedy Search	Best-First	A* Search
Accurate	Faster and more effective		Efficient and can remain optimal under suitable heuristic conditions
Inaccurate	May make poor route decisions		May explore more nodes and become less efficient
Effect on Cost	Ignores actual cost		Considers actual cost
Reliability	Lower		Higher

1.7 Effect of Dynamic Changes

In a flood-affected environment, paths can become blocked unexpectedly.

Effect on Greedy Search:

- Previously selected paths may become unavailable.
- The search may need to be restarted or revised.

- It may select another route without considering accumulated travel cost.

*Effect on A:**

- Updated path costs can be incorporated into replanning.
- Alternative routes can be evaluated using both cost and heuristic information.
- It generally provides a more informed basis for route selection.

1.8 Recommendation

*Recommended Algorithm: A Search**

A* is the most suitable choice because it considers both actual travel cost and estimated distance. For emergency medicine delivery, safety and route quality are more important than selecting the absolute fastest search method.

1.9 Comparison

Feature	Greedy Best-First Search	A* Search
Uses heuristic	Yes	Yes
Uses actual path cost	No	Yes
Speed	High	Moderate
Optimality	Not guaranteed	Guaranteed under suitable conditions
Route-cost awareness	Low	High
Suitability	Quick approximate routing	Cost-aware route planning
Recommendation	No	Yes

1.10 Conclusion

A* Search is the preferred algorithm for the emergency drone because it balances path cost and estimated distance. Although Greedy Best-First Search can provide faster decisions, it may select costly or risky routes. A* provides a more reliable basis for selecting safe and efficient routes in a changing environment.

QUESTION 2

Smart City Traffic Signal Optimization using AI

2.1 Scenario Understanding

A smart city wants to optimize traffic signal timings across hundreds of intersections. The objectives are to reduce waiting time, fuel consumption and traffic congestion. Since traffic conditions continuously change, the optimization system must adapt to different traffic patterns.

2.2 Problem Formulation

The problem can be formulated as an optimization problem where the AI system searches for signal timings that minimize:

- Total vehicle waiting time.
- Fuel consumption.
- Traffic congestion.

The large number of intersections creates a very large search space, making exhaustive traditional search inefficient.

2.3 Hill Climbing

Hill Climbing begins with an initial traffic signal configuration and repeatedly moves to a neighboring configuration that improves the objective value.

Working:

1. Generate an initial signal configuration.
2. Evaluate its traffic performance.
3. Modify one or more signal timings.
4. Accept an improved configuration.
5. Continue until no better neighboring solution is found.

Advantages:

- Simple.
- Fast.

- Requires relatively low memory.

Limitations:

- Can become trapped in a local optimum.
- May have difficulty on plateaus.
- Can struggle with ridges.
- Does not guarantee the global optimum.

2.4 Local Maxima, Plateaus and Ridges**Local Maximum:**

The algorithm reaches a solution that is better than its immediate neighbors but not the best solution overall.

Example: One intersection has excellent traffic flow, but nearby intersections remain congested.

Plateau:

Many neighboring solutions have similar evaluation values, so the algorithm has little information about which direction to move.

Example: Small changes in signal timing produce almost identical traffic performance.

Ridge:

The best improvement may require coordinated changes in multiple directions, which simple Hill Climbing may fail to identify.

Example: Several connected intersections need to change signal timings together.

2.5 Simulated Annealing

Simulated Annealing allows the algorithm to occasionally accept a worse solution, especially during the early stages of search.

This helps the algorithm escape local optima and explore different regions of the search space.

2.6 Genetic Algorithm

A Genetic Algorithm maintains a population of possible traffic signal configurations.

Basic steps:

1. Generate an initial population.
2. Evaluate each solution.
3. Select better solutions.
4. Perform crossover.
5. Apply mutation.
6. Generate a new population.
7. Repeat until a satisfactory solution is obtained.

2.7 Recommended Approach

Recommended Approach: Genetic Algorithm

A Genetic Algorithm is suitable because:

- It can explore a large search space.
- It can maintain multiple candidate solutions.
- It reduces dependence on a single local search path.
- It can optimize multiple traffic objectives.
- It can adapt when traffic conditions change.

2.8 Conclusion

Traffic signal optimization involves a very large and changing search space. Hill Climbing is fast but may become trapped in local optima. Genetic Algorithms provide broader exploration and are therefore a suitable approach for large-scale traffic optimization.

QUESTION 3

Autonomous Mars Rover using Online Search

3.1 Scenario Understanding

An autonomous rover is deployed on Mars to explore unknown terrain, collect samples and avoid hazards such as rocks and craters. Since the rover does not have a complete map, it must learn about its environment while moving.

3.2 Problem Statement

The rover must make safe decisions despite incomplete environmental information. It must continuously update its knowledge and modify its route when new hazards or useful locations are discovered.

3.3 Objectives

- To explore unknown terrain safely.
- To collect scientific samples.
- To avoid environmental hazards.
- To update knowledge continuously.
- To make real-time decisions.

3.4 Need for Online Search

Offline Search requires a sufficiently known problem model before planning.

Online Search allows the agent to:

- Act while searching.
- Observe the environment.
- Learn new information.
- Update its internal model.
- Replan when necessary.

Therefore, online search is more appropriate for the Mars rover.

3.5 Challenges

Partial Observability

The rover cannot observe the complete environment at once. It may only detect nearby obstacles using its sensors.

Dynamic Conditions

Environmental conditions such as dust, terrain changes and newly discovered obstacles may affect navigation.

3.6 Internal Model

The rover can maintain information about:

- Explored areas.
- Safe paths.
- Dangerous regions.
- Obstacles.
- Sample locations.
- Previously visited locations.

Whenever new information is obtained, the internal model is updated.

3.7 Comparison

Search Method	Main Characteristic
Uninformed Search	Searches without heuristic knowledge
Informed Search	Uses heuristic information
Online Search	Learns and updates information while acting

3.8 Recommendation

Recommended Approach: Online Search Agent

It is most suitable because the rover operates without a complete map and must continuously adapt to newly discovered information.

3.9 Conclusion

An Online Search Agent is appropriate for Mars exploration because the rover cannot depend on a complete predefined map. Continuous sensing, learning, model updating and replanning allow it to operate safely in an uncertain environment.

QUESTION 4

University Exam Timetable Generation using Constraint Satisfaction Problem

4.1 Scenario Understanding

A university wants to automatically generate an examination timetable. The system must satisfy student, faculty, room and scheduling constraints while handling an increasing number of courses and students.

4.2 Problem Statement

The objective is to generate a conflict-free timetable that satisfies all required constraints.

4.3 CSP Formulation

A Constraint Satisfaction Problem consists of Variables, Domains and Constraints.

Variables

The variables represent examinations that must be scheduled.

Examples:

- Mathematics
- Physics
- Chemistry
- Biology
- Computer Science

Domains

Domains represent the possible time slots.

Examples:

- Monday Morning
- Monday Afternoon
- Tuesday Morning
- Tuesday Afternoon
- Wednesday Morning

Constraints

The system must satisfy:

1. A student must not have two examinations at the same time.
2. An examination hall cannot conduct two examinations simultaneously.
3. Prerequisite subjects must be scheduled before dependent subjects.
4. Faculty must be available during examinations.
5. Special accommodation requirements must be considered.

4.4 Backtracking Search

Backtracking assigns values to variables one at a time.

Steps:

1. Select an examination.
2. Assign a valid time slot.
3. Check the constraints.
4. If valid, continue with the next examination.
5. If a conflict occurs, return to the previous assignment.
6. Select another available slot.
7. Continue until a valid timetable is obtained.

4.5 Forward Checking

Forward checking improves backtracking by removing values from future variables that would create conflicts.

For example, if two exams share students and one exam is assigned to Monday Morning, that time slot can be removed from the domain of the conflicting exam.

Benefits:

- Reduces the search space.
- Detects conflicts earlier.
- Reduces unnecessary backtracking.
- Improves scheduling efficiency.

4.6 Real-World Challenges

- Large number of students.
- Large number of courses.
- Limited examination halls.
- Faculty availability.
- Student conflicts.
- Special accommodations.
- Last-minute changes.

4.7 Recommended Strategy

Backtracking + Forward Checking

This combination is appropriate because backtracking provides systematic constraint satisfaction while forward checking reduces unnecessary search.

4.8 Conclusion

University examination scheduling is naturally modeled as a CSP. Backtracking with forward checking can efficiently generate valid schedules while handling multiple constraints.

QUESTION 5

Strategic Game AI using Minimax and Alpha-Beta Pruning

5.1 Scenario Understanding

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, evaluate multiple possible moves and respond within strict time limits.

5.2 Problem Statement

The AI must select a strong move from a large game tree while minimizing computation time. The large number of possible moves makes exhaustive game-tree exploration expensive.

5.3 Minimax Algorithm

Minimax models the game using two opposing players:

- **MAX:** AI player, trying to maximize its score.
- **MIN:** Opponent, trying to minimize the AI's score.

The algorithm explores possible future game states and selects the move with the best worst-case outcome.

5.4 Working

1. Generate possible moves.
2. Construct the game tree.
3. Evaluate terminal or depth-limited states.
4. MAX selects the highest evaluation.
5. MIN selects the lowest evaluation.
6. Propagate values toward the root.
7. Select the best available move.

5.5 Computational Challenges

Large game trees create:

- High computational cost.
- Large numbers of possible states.

- Increased memory requirements.
- Longer decision times.
- Difficulty meeting real-time constraints.

5.6 Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

Alpha (α): Best value currently guaranteed for MAX.

Beta (β): Best value currently guaranteed for MIN.

When the search determines that a branch cannot improve the final decision, that branch is pruned.

Advantages:

- Reduces unnecessary computation.
- Allows deeper search within the same time.
- Produces the same Minimax result when applied correctly.
- Suitable for real-time game AI.

5.7 Evaluation Function

A heuristic evaluation function assigns a score to non-terminal game states.

Factor	Purpose
Number of units	Measures military strength
Unit health	Indicates survival capability
Resources	Measures future strategic capability
Territory controlled	Indicates strategic advantage
Distance to enemy	Represents attack opportunity
Remaining time	Supports time-sensitive decisions

A higher evaluation score represents a more favorable position for the AI.

5.8 Recommended Strategy

Minimax with Alpha-Beta Pruning

This approach is recommended because it combines strategic decision-making with efficient search. Alpha-Beta Pruning reduces unnecessary branches, allowing the AI to search deeper within a limited time.

5.9 Conclusion

Minimax provides a systematic method for selecting moves against an intelligent opponent. However, large game trees can make it computationally expensive. Alpha-Beta Pruning improves efficiency by eliminating irrelevant branches while preserving the Minimax decision.